Lab 1.1: The Pattern Audit

Listing 1: Legacy Shopping Cart Implementation

```
// shoppingCart.js - Legacy implementation
var cart = [];
var total = 0;

function addItem(name, price) {
    cart.push({ name: name, price: price });
    total += price;
    updateDisplay();
}

function removeItem(index) {
    var item = cart[index];
    total -= item.price;
    cart.splice(index, 1);
    updateDisplay();
}

function updateDisplay() {
    var cartDiv = document.getElementById('cart');
    cartDiv.innerHTML = '';
    cart.forEach(function(item, index) {
        var itemDiv = document.createElement('div');
        itemDiv.innerHTML = item.name + ' - $' + item.price;
        var removeBtn = document.createElement('button');
        removeBtn.onclick = function() { removeItem(index); };
        itemDiv.appendChild(removeBtn);
        cartDiv.appendChild(itemDiv);
    });
    document.getElementById('total').textContent = '$' + total;
}

function calculateTax(rate) {
    return total * rate;
}

function checkout() {
    var tax = calculateTax(0.08);
    var finalTotal = total + tax;
```

```
    alert('Total: $' + finalTotal);
    cart = [];
    total = 0;
    updateDisplay();
}
```

Task 1

1. Определите все шаблоны проектирования (или прото-шаблоны), которые вы наблюдаете в этом коде.

Наблюдаю вэтом коде признаки прото-паттерна, который пытается имитировать структуру MVC (Model-View-Controller). Это можно увидить в отсутствии инкапсуляции данных и логики, жёсткая привязка между данными, слабая связь "Данные-Вид".

2. Для каждого выявленного шаблона укажите, к какой категории он относится (создание, структура или поведение) на основе главы 6 учебника.

Предпологаю что данный код относится к структурным паттернам. Но из-за отсутствия чёткой структуры и инкапсуляции, это скорее прото-паттерн, чем полноценный шаблон проектирования.

3. Объясните, почему вы считаете, что это настоящий шаблон или нет.

Из-за выше перечисленных причине, я считаю что это не полноценный шаблон проектирования. Поскольку паттерн должен быть переиспользуемым и инкапсулированным, а данный код не соответствует этим критериям.

Task 2

1. Соответствие цели: Успешно ли это решение решает конкретную проблему? Какую проблему оно решает? Приведите доказательства из кода.

Данный код успешно решает поставленную задачу, а именно "управление корзиной покупок". В доказательство можно привести функцию updateDisplay(), котороая вызывается после каждого изменения данных (addItem, removeItem), обеспечивая актуальность UI. Код выполняет свою прямую задачу

2. Полезность: Дает ли это решение доказанную выгоду? Какие преимущества оно предлагает? Есть ли ограничения?

Код даёт кратковременную выгоду в виде простого управления корзиной покупок. Код невозможно тестировать изолированноБ так как он напрямую привязан к DOM-элементам 'cart' и 'total'.

3. Применимость: Применимо ли это решение в широком спектре контекстов или оно специфично для данного сценария? Объясните свое мнение.

В широком контексте данный код не применим, мы не можем использовать его для разных корзин на одной странице, так как глобальные переменные и жёстко пописанные ID элементов вызовут конфликты

Task 3

1. Решает ли оно конкретную проблему? Да, этот код решает проблему с обновлением интерфейса при изменнении состояния массива товаров.

2. Имеет ли оно очевидное решение? Думаю нет. Это самое простое решение через глобальные переменные и прямой манипуляции DOM. Настоящие паттерны предлагают более элегантный, неочевидный с первого взгляда.

3. Описывает ли оно проверенную концепцию? Нет, Прямое манипулирование глобальным состоянием и DOM внутри функции - это скорее "Анти-паттерн" в современном JS, а не проверенная временем эффективная практика.

4. Описывает ли оно взаимосвязь между структурами системы? На мойвзгляд, код слабо описывает отношение. Существует лишь простая связь между данными и функцией отрисовки, но нет чёткого интерфейса взаимодействия.

Task 4

Проанализировав код, я обнаружил что в коде пристутсвуют явные анти-патерны. Во-первых, Gloval Namespace Pollution - переменные var char = [] и var total = 0 доступные любому другому скрипту на старнице. Это приводит к коллизиям имён и трудоуловимыми багами. Во-вторых, Tight Coupling - логика расчёта жестко связана с DOM через document.getElemtById внутри функций обработки данных.

В заключени можно сказать, можно обернуть код в Module Pattern для создания приватной области видимости и передавать зависимости как аргументы.